

Kogna Serial Servo Support — Overview

Kogna can now drive low-cost smart serial-bus servos as ordinary axes: up to **8 servos daisy chained on a single wire**, with position commands broadcast at ~1 kHz and full telemetry (position, speed, load, voltage, temperature, status) flowing back continuously. Two servo families are supported on the same bus hardware, selected by a DSP global (ServoProtocol): the Feetech STS3215 (STS/SCS protocol) and ROBOTIS Dynamixel Protocol 2.0 servos (XL430 etc.) — one family on the bus at a time.

Axes, not peripherals

Serial servos plug into the normal axis machinery via three new modes:

- `SERIAL_SERVO_INPUT_MODE` — axis Position is read from the servo's own magnetic encoder (multi-turn accumulated).
- `SERIAL_SERVO_MODE` — the commanded position is written to the servo, which closes its own loop (analogous to Step/Dir, but sending positions).
- `CL_SERIAL_SERVO_MODE` — same, plus the KFLOP/Kogna servo loop adds a correction on top (analogous to CL Step/Dir), enabling feed-forward compensation of the servo's tracking lag.

The servo's bus ID (0–7) is simply the axis Input/Output Channel number. No FPGA setup is required — the DSP protocol engine claims the bus automatically whenever an axis uses a serial servo mode. Configured axes work with all the existing machinery: coordinated motion, homing, following-error protection, the Step response screen (which now plots serial servo load, current, and voltage), and G-code.

Architecture

The FPGA part (`sts_uart.vhd`) is deliberately simple: a half-duplex 1 Mbaud byte-stream UART with packet buffers on IO0 (or EX_IO_13, freeing IO0/IO1 for hardware encoder 0). **All protocol intelligence lives in the DSP** — packet assembly, checksums, response validation, telemetry decoding, timeouts, and scheduling. Each ~1 ms bus cycle broadcasts one SYNC_WRITE with every actively commanded position (so command rate does not degrade with servo count) and reads full telemetry from one servo round robin. Configuration reads/writes of any servo register go through an aux mailbox that slots between cyclic transactions.

Bring-up and utility C Programs (C Programs\SerialServo)

Programs are provided for the complete life cycle: assigning IDs one servo at a time (`SerialServoSetID.c`), configuring axes (`SerialServoAxisInit.c`, `DxlAxisInit.c`), one-time servo setup (`DxlSetup.c`), dumping and setting every internal servo parameter with per-register documentation (`SerialServoConfig.c`, `DxlConfig.c`), scanning a bus of unknown IDs (`SerialServoScan.c`), layered diagnostics from FPGA identity up through the protocol engine (`SerialServoDiag.c`), raw-bus recovery of a misconfigured servo including factory restore (`SerialServoRecover.c`), and measured-lag tuning tools (`SerialServoTuneLag.c`, `SerialServoLagVsVel.c`, `SerialServoSweepLag.c`, `SerialServoLeadSchedule.c`).

`STS3215_Servos.md` documents the bus architecture, FPGA register map, electrical guidelines, and measured servo characteristics in detail; `SERIAL_SERVO_Axis_Mode.md` documents the axis modes and configuration.

Measured results

- **STS3215**: ~80 ms internal tracking lag (dominated by the servo, not the bus — the command pipeline contributes well under 1 ms). Best tracking uses the velocity-scheduled command-lead compensation tools above.
- **Dynamixel XL430** (tuned recipe baked into `Dx1AxisInit.c`): servo P raised to 10000 plus a constant-time velocity feed forward in `CL_SERIAL_SERVO_MODE` yields **±4 counts (0.35°) tracking at 1500 counts/s** with no preview machinery at all.

Requirements

Serial servo support requires this release's Kogna firmware and FPGA configuration. Servo power comes from its own supply rail (an STS3215 can stall near 2.5 A; size a multi-servo rail accordingly), and the 3.3 V data line guidelines in `STS3215_Servos.md` should be reviewed before first connection.